FlaskGuard

Custom Web Application Firewall

Technical Report – Part 3

Machine Learning Classifier (LightGBM + TF-IDF)
& IP Management System (Blacklist, Strikes, Bans)

Repository:	`github.com/echoenvoy/FlaskGuard`
Report Part:	3 of 5
author:	Abdelmoghit El Asraoui
Key Files:	`ml_classifier.py`, `ip_manager.py`
ML Algorithm:	LightGBM (Gradient Boosting) with Random Forest fallback
Date:	June 2026

Contents

Introduction

The Evasion Problem

Part 2's regex detection engine is effective against known attack patterns but fundamentally brittle against evasion. An attacker who knows the ruleset can craft payloads that carry the same malicious intent while avoiding every pattern. Classic evasion techniques include:

- **Obfuscation:** `S%45LECT` instead of `SELECT` (URL encoding)
- **Comment injection:** `SEL/**/ECT` (SQL inline comments)
- **Case variation:** `sElEcT` instead of `SELECT`
- **Novel payloads:** Entirely new attack patterns not yet in the ruleset

A machine learning classifier trained on the *statistical properties* of attack payloads can generalise beyond specific patterns, catching obfuscated or novel inputs that share the same underlying character-level statistics as known attacks.

The Repeat Offender Problem

Part 2's logging system records attack events but does not act on the pattern of *repeated attacks from the same IP*. A determined attacker sending hundreds of probes should face progressively increasing consequences, culminating in a temporary or permanent ban. The IP management system addresses this through a strike-based mechanism.

Scope

- **ml_classifier.py:** LightGBM classifier with Random Forest fallback, character n-gram TF-IDF features, semantic heuristic fallback (`_semantic_attack_confidence`), training/retraining pipelines, persistence, prediction API, feedback integration, 5-fold cross-validation, gain-based feature importance
- **ip_manager.py:** Strike recording, auto-ban threshold logic, ban duration management, manual ban/unban, blacklist persistence
- **config.py:** YAML-driven configuration for all subsystems (ML threshold, rate limits, ban parameters, proxy settings, storage paths)
- **app.py:** Two-payload inspection pipeline — regex receives full request data including URL path, ML receives only user-controllable values
- Integration of all systems with the reverse proxy inspection pipeline

Inspection Pipeline Architecture

FlaskGuard's request inspection uses a three-layer pipeline with separate payload routing for different detectors:

Layer	Input	Detector
File Scanner	Upload filename + content	Extension/MIME blacklist
Regex Engine	Full path + query + form + JSON	5 attack categories (SQLi, XSS, traversal, CMDi, file injection)
ML + Semantic	User-controlled values only	LightGBM + `_semantic_attack_confidence`

Critical design decision: The regex engine receives a combined payload that includes `request.full_path` (the raw URL with query string), allowing it to detect path traversal and URL-encoded attacks in the URL itself. The ML classifier receives only the user-controlled *values* (query parameters, form fields, JSON body) without URL syntax, preventing false positives from characters like `/`, `?`, `=` that appear in all URLs but are strong attack indicators in the n-gram model.

Listing 1: Separate payload assembly for regex and ML

```
# Regex payload: includes full URL path for traversal detection
regex_parts = [request.full_path, *path_parts, *user_parts]
regex_payload = " ".join(str(part) for part in regex_parts)

# ML payload: user-controlled values only, no URL syntax
ml_payload = " ".join(str(part) for part in [*user_parts, *path_parts
    [1:]])
```

All thresholds and parameters are externalised to `config.yml`, enabling operators to tune the WAF without modifying source code:

Listing 2: Relevant config.yml excerpt

```
ml_classifier:
  enabled: true
  confidence_threshold: 0.85

ip_blacklist:
  enabled: true
  threshold_strikes: 5
  ban_duration_minutes: 60
```

Machine Learning Classifier – `ml_classifier.py`

Algorithm Evolution: From Naive Bayes to LightGBM

FlaskGuard's ML classifier evolved through three generations, each addressing specific limitations of its predecessor:

Phase	Algorithm	Accuracy	False Positive Rate
Phase 1 (Initial)	Multinomial Naive Bayes	89.08%	13.13%
Phase 2 (Upgrade)	Random Forest (100 trees)	98.32%	0.00%
Phase 3 (Current)	**LightGBM (gradient boosting)**	**98%+**	**<0.5%**

Why Naive Bayes Was Insufficient

The initial MultinomialNB model suffered from its fundamental independence assumption. It treats every character n-gram as an independent feature, unable to learn interactions between them. For example, the n-gram `-` appears in both SQL injection (`' OR 1=1 -`) and benign text (`meeting at 3pm - bring notes`). NB gives both contexts the same probability contribution, resulting in an unacceptable 13% false positive rate — roughly 1 in 8 benign requests blocked.

Why LightGBM Was Selected

LightGBM was chosen as the production model over alternatives for three WAF-critical reasons:

- **Native sparse matrix support:** TF-IDF with 400 character n-grams produces a feature matrix where >95% of values are zero. Random Forest internally densifies this (wasting RAM and slowing inference). LightGBM processes sparse matrices natively, achieving **sub-millisecond inference (0.1–0.5ms)**.
- **Leaf-wise tree growth:** Unlike level-wise boosting (XGBoost default), LightGBM grows trees by splitting the leaf with the highest loss reduction first. This produces more accurate models with fewer nodes, particularly on small datasets.
- **Built-in regularisation:** L1 (`reg_alpha`) and L2 (`reg_lambda`) penalties are critical for a model trained on only ~630 samples. Without them, gradient boosting overfits severely — memorising rare character sequences instead of learning general patterns.

How Gradient Boosting Works

Unlike Naive Bayes (which computes posterior probabilities from feature frequencies) or Random Forest (which averages independent decision trees), LightGBM builds trees **sequentially**, each new tree correcting the errors of the previous ensemble:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta \cdot f_t(x_i) \tag{1}$$

where $\hat{y}_i^{(t)}$ is the prediction after t trees, η is the learning rate (0.08 in FlaskGuard's configuration), and $f_t(x_i)$ is the t-th tree's prediction on input x_i. Each tree minimises the gradient of the loss function with respect to the current predictions:

$$g_i = \frac{\partial L(y_i, \hat{y}_i)}{\partial \hat{y}_i} \tag{2}$$

This sequential error correction allows LightGBM to learn complex feature interactions that Naive Bayes misses entirely and Random Forest captures less efficiently.

Graceful Fallback Architecture

The code auto-detects whether LightGBM is installed and degrades to Random Forest:

Listing 3: Classifier factory with fallback

```
try:
    from lightgbm import LGBMClassifier
    _HAS_LIGHTGBM = True
except ImportError:
    _HAS_LIGHTGBM = False

def _make_classifier():
    if _HAS_LIGHTGBM:
        return LGBMClassifier(
            n_estimators=100, max_depth=5,
```

```
            reg_alpha=2.0, reg_lambda=2.0, ...)
    return RandomForestClassifier(
        n_estimators=100, max_depth=15, ...)
```

LightGBM installed	LGBMClassifier
LightGBM missing	RandomForestClassifier
Old NB model on disk	Auto-replaced on next retrain

The Overfitting Problem and Its Solution

After switching to LightGBM with 5,000 features, the model exhibited severe overfitting. Random keyboard gibberish containing semicolons, commas, and colons was classified as an attack with **95.34% confidence**:

```
Input:  "jdghudn,sufbsufhus,s fusds;d:vsijuhsufbsudcisfdsnf"
Result: BLOCKED (false positive) - 95.34% confidence
```

Root Cause

With 5,000 features on 580 samples (approximately 8.6 features per sample), the model had room to memorise, not generalise. The training data had a critical blind spot:

Character	Attack Samples	Normal Samples
; (semicolon)	45	0
: (colon)	30	0
, (comma)	25	0

The model learned that `;`, `:`, and **`,` always indicate attacks**, because it had never seen them in a benign context.

Four-Pronged Fix

1. **Feature reduction:** `max_features` was reduced from 5,000 to 400. Rare n-grams like `;d` (from the gibberish) no longer make the top 400 features, forcing the model to rely on frequent patterns that generalise.
2. **Stronger regularization:** `reg_alpha` and `reg_lambda` increased from 0.1 to 2.0. `max_depth` reduced from 12 to 5. `min_child_samples` increased from 5 to 20. L1 zeros out weak feature coefficients; L2 prevents any single feature from dominating; shallow trees prevent complex chains of rare features.
3. **Diverse normal samples:** 50 new benign training samples were added containing `;`, `:`, `,`, `{`, `}` in harmless contexts: `"hello; world"`, `"colors: red, green"`, `"subject: meeting at 3pm"`, `"csv: id,name,email"`. The model now has counter-examples for every special character.
4. **Higher confidence threshold:** Raised from 70% to **85%**. The regex engine already catches obvious attacks deterministically. The ML model is a fallback for novel/obfuscated attacks — it should only fire when it is very confident.

Result: The gibberish confidence dropped from 95.34% to **74.37%** — safely below the 85% threshold. Meanwhile, real attack detection remained unaffected:

```
"' OR 1=1 --"                    99.6% -> BLOCKED
"<script>alert(1)</script>"     100.0% -> BLOCKED
"../../etc/passwd"               99.0% -> BLOCKED
"; ls -la"                       98.5% -> BLOCKED
```

Current LightGBM Configuration

Parameter	Value
Algorithm	LGBMClassifier (gradient boosting)
Number of trees	100
Max depth	5
Number of leaves	31
Learning rate	0.08
L1 regularisation (`reg_alpha`)	2.0
L2 regularisation (`reg_lambda`)	2.0
Min samples per leaf	20
Subsample rate	0.8 (80% of rows per tree)
Column sample rate	0.6 (60% of features per tree)
Class weighting	`balanced` (automatic)
Fallback if uninstalled	RandomForestClassifier

Feature Extraction: Character N-gram TF-IDF

The choice of **character-level n-grams** as features is the most distinctive and important technical decision in the classifier design.

Why Character N-grams?

Word-level tokenisation breaks on attack payloads, which typically contain special characters, SQL keywords without spaces, and intentionally mangled text. Character n-grams are *token-agnostic*: they extract overlapping substrings of length n from the raw payload string, capturing statistical regularities at the character level.

Examples of character 3-grams extracted from the payload `' OR 1=1`:

`' O`, ` OR`, `OR `, `R 1`, ` 1=`, `1=1`

Attack payloads consistently produce higher frequencies of n-grams like `OR `, `1=1`, `SE`, `LE`, `EC`, `CT` (from UNION SELECT), `<sc`, `cri`, `rip`, `ipt` (from `<script>`), and `../` (path traversal).

The `char_wb` Analyzer

FlaskGuard uses the `char_wb` (characters within word boundaries) analyzer rather than `word` or `char`. With `char_wb`, n-grams are extracted from characters inside whitespace-delimited tokens but **not** across whitespace boundaries. This means an n-gram like `;d` from the gibberish string `"fusds;d:vsijuhsufbsudcisfdsnf"` IS captured (since there are no spaces between the semicolon and surrounding chars), while meaningless n-grams spanning unrelated words are excluded.

TF-IDF Weighting

Raw n-gram counts are weighted using TF-IDF (Term Frequency – Inverse Document Frequency):

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \cdot \log \frac{N}{|\{d : t \in d\}|} \tag{3}$$

TF-IDF up-weights n-grams that are distinctive to a particular class and down-weights n-grams that appear in most payloads (such as `th`, `he`, which are common in English prose but also in benign search queries).

Vectoriser Configuration

Listing 4: TF-IDF vectoriser configuration

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(
    analyzer='char_wb',       # Character-level, within word boundaries
    ngram_range=(2, 4),       # Extract 2-, 3-, and 4-grams
    max_features=400,         # Cap vocabulary to prevent overfitting
)
```

The `ngram_range=(2, 4)` setting captures:

- **2-grams:** Detect SQL operators (`OR`, `=1`, `-`)
- **3-grams:** Detect keywords (`SEL`, `ECT`, `scr`)
- **4-grams:** Detect longer patterns (`SELE`, `NION`, `crip`)

The `max_features=400` limit was chosen after extensive experimentation to balance accuracy against overfitting — the final value was arrived at by progressively reducing from 5,000 through 800 until false positives on benign inputs with special characters were eliminated.

Training Data

The initial training dataset is assembled from multiple sources. The baseline corpus now contains **630 samples** (200 normal, 430 attack) across 10 attack categories:

Source	Label	Description
Baseline attack corpus	attack	Curated collection of SQLi, XSS, traversal, CMDi, RFI, NoSQL, XXE, LDAP, SSTI
Baseline benign corpus	benign	Normal text PLUS 50 diverse samples with `;`, `:`, `,` in harmless contexts
`attacks.json` log	attack	Payloads captured by FlaskGuard in production (capped at 500, deduplicated)
Admin feedback	both	Operator-labelled corrections from `feedback.json` (capped at 1,000 entries)

Critical design decision: The 50 diverse normal samples were added specifically to prevent the model from learning that special characters (`;`, `:`, `,`, `{`, `}`) always indicate

attacks. Without these counter-examples, the model developed a severe false positive bias against any input containing these characters.

Model Training Pipeline

Listing 5: Training pipeline with LightGBM

```python
from lightgbm import LGBMClassifier
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer

def train_classifier(payloads, labels):
    pipeline = Pipeline([
        ('tfidf', TfidfVectorizer(
            analyzer='char_wb',
            ngram_range=(2, 4),
            max_features=400
        )),
        ('clf', LGBMClassifier(
            n_estimators=100,
            max_depth=5,
            num_leaves=31,
            learning_rate=0.08,
            min_child_samples=20,
            subsample=0.8,
            colsample_bytree=0.6,
            reg_alpha=2.0,
            reg_lambda=2.0,
            class_weight='balanced',
            random_state=42,
            verbosity=-1
        ))
    ])
    pipeline.fit(payloads, labels)
    return pipeline
```

The retraining pipeline additionally runs 5-fold stratified cross-validation to validate generalisation:

Listing 6: Cross-validation

```python
from sklearn.model_selection import cross_val_score
cv_scores = cross_val_score(pipeline, texts, labels, cv=5, scoring='f1')
cv_mean = round(cv_scores.mean() * 100, 2)  # e.g., 92.93%
```

Model Persistence

The trained model is serialised to two files:

- `classifier.pkl`: The scikit-learn `Pipeline` object (vectoriser + classifier), serialised with `pickle`. The pipeline includes both the fitted TF-IDF vectoriser and the trained LightGBM model.
- `model_meta.json`: A human-readable metadata file with training statistics (accuracy, precision, recall, F1, 5-fold CV scores, sample counts per class, training timestamp, algorithm name).

On startup, if the model file exists, it is loaded from disk. If it does not exist (fresh installation), the classifier trains from the baseline dataset before the first request is processed.

Prediction API

The classifier uses a two-stage confidence-gated prediction interface that combines ML model probability with heuristic rules:

Listing 7: Two-stage detection with semantic fallback

```python
def ml_detect(input_data: str) -> bool:
    if not input_data or len(input_data) < 3:
        return False
    return ml_confidence(input_data) >= 85.0

def ml_confidence(input_data: str) -> float:
    try:
        model = get_model()
        proba = model.predict_proba([input_data])[0]
        model_conf = round(float(proba[1]) * 100, 1)
        semantic_conf = _semantic_attack_confidence(input_data)
        return max(model_conf, semantic_conf)
    except Exception:
        return _semantic_attack_confidence(input_data)
```

Key design decision: The final confidence is the `max` of the LightGBM model probability and the semantic heuristic score. This means if either system is confident that a payload is malicious, it will be blocked. The semantic fallback catches attacks that the statistical model might miss due to training data gaps.

Semantic Attack Confidence

FlaskGuard includes a lightweight heuristic fallback (`_semantic_attack_confidence`) that catches specific attack families the regex engine or ML model might miss. Rather than relying on regex patterns (which requires maintaining an ever-growing ruleset) or the statistical model (which depends on training data coverage), this function uses keyword-based pattern matching for known high-confidence attack indicators:

Listing 8: Semantic confidence heuristics

```python
def _semantic_attack_confidence(input_data: str) -> float:
    text = str(input_data).lower()

    # Template injection (Jinja2, Twig, ERB, JSP, etc.)
    if "{{" in input_data and "}}" in input_data:
        return 96.0
    if "${" in input_data and "}" in input_data:
        return 94.0
    if "__globals__" in input_data or "__subclasses__" in input_data:
        return 98.0

    # Complex attack keyword groups (NoSQL, LDAP, SSTI chains)
    phrase_groups = [
        ("mongodb", "where", "function", "return", "true"),
        ("password", "regex", "wildcard", "login", "bypass"),
```

```
        ("ldap", "uid", "wildcard", "objectclass"),
        ("constructor", "constructor", "alert"),
        ("template", "constructor", "prototype", "execution"),
        ("server", "side", "template", "globals"),
        ("config", "items"),
    ]
    for group in phrase_groups:
        if all(term in text for term in group):
            return 94.0
    return 0.0
```

Why this approach: The semantic fallback bridges the gap between the regex engine (which only covers 5 categories with explicit patterns) and the ML model (which must learn from examples). It catches:

- **Template injection (SSTI):** `{{config.items()}}`, `{{self.__init____globals__}}`, Jinja2 chains — patterns that may appear in URL paths without explicit operators
- **NoSQL injection:** MongoDB `$where`, `$gt`, `$regex` bypass phrases — often written as JSON-like text without SQL symbols
- **LDAP injection:** `uid`, `objectclass` wildcard bypass attempts
- **JavaScript/Angular prototype pollution:** `constructor.constructor` chains

The semantic confidence returns fixed high scores (94.0–98.0%) rather than attempting to model probability distributions. These scores are well above the 85% threshold, ensuring immediate blocking without false positives — the matched patterns are virtually never seen in legitimate traffic.

The confidence threshold of **0.85** (raised from the original 0.70) is critical: the regex engine already catches obvious attacks deterministically. The ML model and semantic fallback are a second and third line of defence for novel or obfuscated attacks — they should only fire when very confident. This higher threshold eliminated false positives on benign inputs containing special characters while maintaining strong detection of real attacks.

Admin API – ML Endpoints

The admin dashboard exposes four ML-related API endpoints:

Endpoint	Method	Description
`/admin/api/ml/status`	`GET`	Returns model metadata: algorithm type, accuracy, precision, recall, F1, 5-fold CV scores, sample counts, training timestamp
`/admin/api/ml/retrain`	`POST`	Triggers a full retrain from baseline + attack logs + feedback; returns validation metrics
`/admin/api/ml/feedback`	`POST`	Accepts an admin-labelled correction (`{"payload": "...", "is_attack": false}`)

Endpoint	Method	Description
`/admin/api/ml/features`	`GET`	Returns top N character n-grams ranked by gain-based feature importance from LightGBM

Feature Importances Endpoint

The `/admin/api/ml/features` endpoint returns the character n-grams with the highest **gain-based** importance scores. Unlike Naive Bayes (which used log-probability differences) or Random Forest (which uses mean decrease in impurity), LightGBM's gain-based importance measures the *total reduction in loss* contributed by splits on each feature across all trees:

Listing 9: Gain-based feature importance extraction

```python
def get_feature_importance(top_n=20):
    model = get_model()
    clf = model.named_steps['clf']
    vectorizer = model.named_steps['tfidf']
    feature_names = vectorizer.get_feature_names_out()

    # Prefer gain-based importance for LightGBM
    if hasattr(clf, 'booster_'):
        importances = clf.booster_.feature_importance(
            importance_type='gain')
    else:
        importances = clf.feature_importances_

    indices = importances.argsort()[::-1][:top_n]
    return [{"feature": feature_names[i],
             "importance": float(importances[i])}
            for i in indices]
```

When run on the trained model, the top features confirm the model has learned legitimate attack indicators rather than spurious correlations:

Rank	N-gram	Attack Context
1	'	SQL quote delimiter
2	-	SQL comment (bypass authentication)
3	;	Command / query separator
4	<	HTML/SVG tag opener
5	/	Path separator (traversal)
6	:	URL scheme / NoSQL operator

This endpoint provides model transparency: operators can inspect *what* the model has learned to associate with attacks, which builds trust and helps identify potential biases in the training data.

Retraining Workflow

The retraining endpoint triggers a full pipeline retrain with validation:

1. Load baseline attack and benign payloads from the built-in corpus (~630 samples).

2. Load all attack payloads from `attacks.json` (live capture, capped at 500, deduplicated).
3. Load all feedback entries from `feedback.json` (admin corrections, capped at 1,000).
4. Deduplicate and merge into a combined training set.
5. Stratified train/test split (80/20) — ensures both classes are represented proportionally in each split.
6. Fit the TF-IDF vectoriser + LightGBM classifier on the training split.
7. Compute validation metrics (accuracy, precision, recall, F1) on the hold-out test split.
8. Run 5-fold stratified cross-validation on the full dataset to measure generalisation.
9. Atomically swap the live model (thread-safe via `threading.Lock`).
10. Persist the new model to `classifier.pkl` and update `model_meta.json` with all metrics.

The atomic model swap is important: operators can retrain and deploy an improved model during an active attack campaign without taking the WAF offline. The `threading.Lock` ensures no request sees a partially-written model file.

IP Management System – `ip_manager.py`

Overview

The IP management system implements a tiered response to repeated attacks from the same IP address:

Event	Action	Duration
Single attack	Record strike	–
Strikes $\geq$ threshold (default: 5)	Auto-ban	Default: 60 minutes
Manual ban via admin dashboard	Immediate ban	Configurable / permanent

Strike System

Each attack event recorded by the logger also calls `ip_manager.record_strike(ip, attack)`. The strike counter for that IP is incremented:

Listing 10: Strike recording logic

```
def record_strike(ip: str, attack: dict):
    """
    Record an attack strike for an IP.
    Auto-bans the IP if the strike threshold is exceeded.
    """
    if ip not in strikes:
        strikes[ip] = {"count": 0, "attacks": []}

    strikes[ip]["count"] += 1
    strikes[ip]["attacks"].append({
        "type": attack["attack_type"],
```

```
        "timestamp": datetime.utcnow().isoformat()
    })

    threshold = config["ip_blacklist"]["threshold_strikes"]
    if strikes[ip]["count"] >= threshold:
        ban_ip(ip, reason="auto_ban", duration_minutes=
               config["ip_blacklist"]["ban_duration_minutes"])
```

Blacklist Data Structure

The blacklist is maintained as a Python dictionary in memory and persisted to `blacklist.json`:

Listing 11: Blacklist entry schema

```
{
    "203.0.113.42": {
        "banned_at": "2026-06-09T14:30:00Z",
        "expires_at": "2026-06-09T15:30:00Z",  # null for permanent bans
        "reason": "auto_ban",                    # or "manual"
        "strike_count": 7,
        "ban_type": "temporary"                  # or "permanent"
    }
}
```

Ban Check in the Inspection Pipeline

The IP blacklist check runs as the *second* step in the inspection pipeline (after admin auth, before file scanning), ensuring that banned IPs receive a 403 immediately without consuming any detection resources:

Listing 12: Blacklist check in before$_r$*equesthook*

```
@app.before_request
def check_blacklist():
    ip = request.remote_addr
    if request.path.startswith('/admin'):
        return  # Admin routes bypass blacklist check

    if ip_manager.is_banned(ip):
        return jsonify({"error": "Your IP is blocked."}), 403
```

Ban Expiry Mechanism

Temporary bans include an `expires_at` timestamp. The `is_banned()` function checks whether the current UTC time is before the expiry:

Listing 13: Ban expiry check

```
def is_banned(ip: str) -> bool:
    if ip not in blacklist:
        return False

    entry = blacklist[ip]
    if entry.get("expires_at") is None:
        return True  # Permanent ban

```

```
    expiry = datetime.fromisoformat(entry["expires_at"])
    if datetime.utcnow() > expiry:
        # Ban has expired; remove from blacklist
        del blacklist[ip]
        save_blacklist()
        return False

    return True
```

Expired bans are removed lazily (on the next check for that IP) rather than via a background cleanup task, keeping the implementation simple and single-threaded.

Manual Ban and Unban

Operators can manually ban or unban any IP via the admin dashboard or directly via the API:

Listing 14: Manual ban and unban via curl

```
# Manual ban
curl -u admin:admin123 \
  -X POST http://localhost/admin/api/ban \
  -H "Content-Type: application/json" \
  -d '{"ip": "198.51.100.5", "reason": "known scanner"}'

# Unban
curl -u admin:admin123 \
  -X POST http://localhost/admin/api/unban \
  -H "Content-Type: application/json" \
  -d '{"ip": "198.51.100.5"}'
```

Manual bans are permanent by default (no `expires_at`). An optional `duration_minutes` parameter can create a timed manual ban.

Blacklist Persistence

The blacklist is written to `blacklist.json` on every modification (ban/unban). On application startup, the file is loaded and any expired entries are pruned. This ensures that bans survive application restarts – a critical operational requirement.

Listing 15: Blacklist load and prune on startup

```
def load_blacklist():
    """Load blacklist from disk, pruning expired entries."""
    if not os.path.exists(blacklist_path):
        return {}

    with open(blacklist_path) as f:
        data = json.load(f)

    # Prune expired bans
    now = datetime.utcnow()
    active = {
        ip: entry for ip, entry in data.items()
        if entry.get("expires_at") is None or
           datetime.fromisoformat(entry["expires_at"]) > now
    }
```

```
16     return active
```

Dashboard Integration

The admin dashboard displays the blacklist via the `/admin/api/blacklist` endpoint. Each banned IP entry shows:

- IP address
- Ban reason (auto/manual)
- Ban timestamp
- Expiry time or "Permanent"
- Strike count (number of detected attacks before the ban)
- Unban button

Integration – ML Classifier and IP Manager in the Pipeline

Full Detection Pipeline with ML

With both the ML classifier and IP manager in place, the complete pipeline is:

1. **Admin auth check:** Validate credentials for `/admin` routes.
2. **IP blacklist check:** If IP is banned, return 403 immediately.
3. **File upload scan** (Part 4): Inspect uploaded files if present.
4. **Regex detection:** Apply 5 attack pattern categories (SQLi, XSS, traversal, CMDi, file injection) to the full request including URL path.
5. **ML classifier fallback:** If regex passes, run LightGBM + semantic heuristics on user-controlled values only. If confidence ≥ 0.85, classify as attack.
6. **Log and strike:** If attack detected (by either layer), log event and increment strike counter.
7. **Auto-ban check:** If strikes $\geq$ threshold, add IP to blacklist.
8. **Forward or block:** Return 403 if attack; proxy request to backend if clean.

Detection Method Attribution

The `detection_method` field in the attack log distinguishes which layer caught the attack:

Value	Meaning
`regex`	A regex pattern matched the payload
`ml_classifier`	LightGBM or semantic heuristic classified with confidence ≥ 0.85

This attribution allows operators to track the relative contribution of each detection layer over time, and to tune thresholds based on observed false-positive and false-negative rates.

Conclusion

Part 3 of FlaskGuard adds two intelligent security subsystems that significantly extend the WAF's capability beyond the deterministic regex detection of Part 2. The LightGBM gradient boosting classifier with character n-gram TF-IDF features, combined with a semantic heuristic fallback, provides a statistically-grounded and rule-aware detection layer that can identify novel, obfuscated, and semantic-level attack payloads that evade specific regex patterns. The feedback-driven retraining pipeline creates a continuous improvement loop, allowing the model to adapt to new attack patterns observed in production without manual rule updates.

The IP management system implements an automated deterrence mechanism: attackers who persist beyond a configurable strike threshold are automatically banned for a configurable duration, while operators retain full manual control via the admin dashboard API. Together, these systems represent a meaningful step toward an adaptive, learning WAF – one that becomes more effective with time and use, rather than requiring constant manual rule maintenance.